

Phase 1: Hashing Foundations — From Searching to Collisions

1. Main Goal of Phase 1

Phase 1 teaches the basic idea of hashing.

Before learning advanced hash table methods, you must first understand:

- Why hashing is needed
- What a hash table is
- What a key is
- What an index is
- What a hash function does
- How ideal hashing works
- Why ideal hashing wastes memory
- How modulus hashing works
- What a collision is
- Why collision handling is needed
- The two main collision-resolution families

The main beginner idea is:

Hashing tries to find data by calculating its position directly.

Instead of searching many places one by one, hashing tries to jump directly to the correct place.

2. Why Searching Can Be Slow

Searching means finding whether a value exists in a collection.

Example array:

[8, 3, 6, 10, 15, 18, 4]

If we want to find 15, we need a searching method.

3. Linear Search

What linear search means

Linear search checks values one by one from the beginning.

Example:

8 → 3 → 6 → 10 → 15

It finds 15, but it had to check many values first.

Worst-case time

If there are N values, linear search may need to check all N values.

So the worst-case time is:

$O(N)$

Simple meaning of $O(N)$

$O(N)$ means:

As the number of values increases, the search time also increases.

If the array is small, linear search is okay.

If the array is large, linear search can become slow.

4. Binary Search

What binary search means

Binary search is faster than linear search.

Its time complexity is:

$O(\log N)$

This is much faster than $O(N)$ for large data.

Important condition

Binary search only works if the data is sorted.

Example of sorted data:

```
[3, 4, 6, 8, 10, 15, 18]
```

If the data is not sorted, binary search will not work correctly.

Problem with binary search

To use binary search, the data must be sorted first.

Sorting takes extra work.

Also, if we keep inserting new values, we may need to maintain sorted order.

That can also take extra effort.

5. Why Hashing Is Introduced

Hashing tries to make searching faster.

Instead of asking:

```
Where is this value? Let me search one by one.
```

Hashing asks:

```
Can I calculate where this value should be?
```

If we can calculate the correct position directly, searching can become very fast.

The average goal of hashing is:

```
 $O(1)$ 
```

Simple meaning of $O(1)$

$O(1)$ means constant time.

It means the operation does not depend much on the number of values.

Example:

Go directly to the correct position.

6. Real-Life Analogy: Locker Example

Imagine a university locker system.

Suppose each student has a student ID, and the locker number is the same as the student ID.

Example:

Student ID 18 → Locker 18

To find student 18's locker, you do not check locker 0, locker 1, locker 2, and so on.

You go directly to locker 18.

This is the basic idea of hashing.

7. Basic Hashing Vocabulary

7.1 Key

A key is the value we want to store or search.

Example:

15

If we are storing student IDs, the student ID can be the key.

If we are storing phone numbers, the phone number can be the key.

7.2 Hash Table

A hash table is the array where values are stored.

Example:

Index:	0	1	2	3	4
Value:	—	—	—	—	—

The hash table stores keys at calculated positions.

7.3 Index

An index is the position inside the array.

Example:

array[3]

Here, **3** is the index.

7.4 Hash Function

A hash function is a rule or formula that changes a key into an index.

General idea:

key → hash function → index

Example:

15 → H(15) → 5

This means key **15** is stored at index **5**.

7.5 Domain / Key Space

The domain, or key space, means all possible input keys.

Example:

If student IDs can be from 0 to 9999, then the key space is:

0 to 9999

7.6 Range

The range means all possible output indices.

Example:

If the hash table size is 10, valid indices are:

0 to 9

So the range is:

0, 1, 2, 3, 4, 5, 6, 7, 8, 9

8. The Hashing Process

Hashing follows this process:

Key → Hash Function → Index → Store/Search in Hash Table

Example:

Key = 15
Hash function = $H(x)$
Index = $H(15)$

So, if:

$$H(15) = 5$$

then key 15 should be stored at index 5.

9. Ideal Hashing / Direct Addressing

9.1 What ideal hashing means

In ideal hashing, the key itself becomes the index.

Formula:

$$H(x) = x$$

This means:

Key 8 → Index 8
Key 15 → Index 15
Key 3 → Index 3

This is also called direct addressing.

9.2 Why it is called direct addressing

It is called direct addressing because the key directly gives the array address.

Example:

Key 10 → Go directly to index 10

No searching is needed.

9.3 Example of ideal hashing

Suppose we have these keys:

8, 3, 6, 10, 15, 18, 4

Using:

$$H(x) = x$$

we store them like this:

Key	Index
8	8
3	3
6	6
10	10
15	15
18	18
4	4

So the key and index are the same.

9.4 Searching in ideal hashing

If we want to search for 10, we calculate:

$$H(10) = 10$$

Then we directly check index 10.

If index 10 contains 10, the key is found.

This is very fast.

9.5 Time complexity of ideal hashing

Insertion:

$O(1)$

Search:

$O(1)$

This is because we directly calculate the position.

10. Problem with Ideal Hashing

Ideal hashing is fast, but it has a big problem.

The problem is memory waste.

10.1 Why memory is wasted

Suppose the keys are:

8, 3, 6, 10, 15, 18, 4

The largest key is 18.

So we need an array that has indices from 0 to 18.

That means we need 19 positions.

This is still acceptable.

But suppose a new key arrives:

100

Now we need an array with indices from 0 to 100.

That means we need 101 positions.

But we may only be storing a few keys.

Many array positions will stay empty.

That wastes memory.

10.2 Main weakness of direct addressing

Direct addressing depends on the largest possible key.

Not on the number of actual stored keys.

This can be a serious space problem.

Example:

Keys stored: 8, 15, 100

Only 3 keys are stored.

But the array needs at least 101 positions.

That is not efficient.

11. Modulus Hashing

To avoid huge arrays, we use a fixed table size.

Example:

Table size = 10
Valid indices = 0 to 9

Now we need a hash function that always gives an index between 0 and 9.

A common method is modulus hashing.

Formula:

$H(x) = x \bmod \text{size}$

If the table size is 10:

$$H(x) = x \bmod 10$$

12. What Does Mod Mean?

`mod` means remainder after division.

Example:

$$15 \bmod 10 = 5$$

Because:

$$15 \div 10 = 1 \text{ remainder } 5$$

So:

$$15 \bmod 10 = 5$$

That means key `15` maps to index `5`.

13. Why Modulus Keeps the Index Inside the Table

If table size is 10, valid indices are:

0 to 9

When we calculate:

$$\text{key} \bmod 10$$

The answer will always be from `0` to `9`.

Examples:

$$\begin{aligned} 8 \bmod 10 &= 8 \\ 10 \bmod 10 &= 0 \end{aligned}$$

```
15 mod 10 = 5
25 mod 10 = 5
99 mod 10 = 9
```

So modulus helps us keep the index inside the valid array range.

14. Modulus Hashing Dry Run

Use this hash function:

```
H(x) = x mod 10
```

Insert these keys:

```
8, 3, 6, 10, 15
```

Key	Calculation	Index
8	8 mod 10	8
3	3 mod 10	3
6	6 mod 10	6
10	10 mod 10	0
15	15 mod 10	5

The hash table becomes:

```
Index 0: 10
Index 1: empty
Index 2: empty
Index 3: 3
Index 4: empty
Index 5: 15
Index 6: 6
Index 7: empty
Index 8: 8
Index 9: empty
```

15. Collision

15.1 What a collision means

A collision happens when two or more keys map to the same index.

Example:

```
15 mod 10 = 5
25 mod 10 = 5
```

Both 15 and 25 want index 5.

But one array position cannot normally store two values directly.

So this is a collision.

15.2 Collision example with key 25

We already inserted key 15 at index 5.

Now try to insert key 25.

Calculation:

```
25 mod 10 = 5
```

But index 5 already contains 15.

So:

```
15 and 25 both want index 5
```

This is a collision.

15.3 Many-to-one mapping

In ideal hashing, the mapping can be one-to-one.

Example:

```
8 → 8  
15 → 15  
25 → 25
```

Each key gets its own index.

But in modulus hashing, many keys can go to the same index.

Example:

```
15 → 5  
25 → 5  
35 → 5  
45 → 5
```

This is called many-to-one mapping.

Many different keys map to one index.

16. Why Collisions Are Important

Modulus hashing solves the memory problem, but it creates collisions.

So hashing has two parts:

1. Use a hash function to find an index.
2. If a collision happens, use a collision-resolution method.

Without collision handling, a hash table is incomplete.

If two keys want the same position, the program must know what to do.

That is why the next phases teach collision-resolution methods.

17. Two Main Collision-Resolution Families

There are two major families:

1. Open hashing / Chaining
2. Closed hashing / Open addressing

18. Open Hashing / Chaining

18.1 Basic idea

In open hashing, also called chaining, we are allowed to use extra memory outside the original array.

Usually, each table index points to a linked list.

If multiple keys map to the same index, they are stored in the linked list at that index.

Example:

Index 5: 15 → 25 → 35

Here, keys 15, 25, and 35 all map to index 5.

Instead of replacing each other, they are connected in a chain.

18.2 Why chaining is easy to understand

Chaining is visual.

If a collision happens, we simply add the new key to the list at that bucket.

This will be studied in Phase 2.

19. Closed Hashing / Open Addressing

19.1 Basic idea

In closed hashing, all keys must stay inside the original hash table array.

No linked lists are used.

If the calculated index is full, the algorithm searches for another empty index inside the same table.

Example:

```
25 wants index 5
Index 5 is full
Try another index inside the same table
```

19.2 Why it is also called open addressing

It is called open addressing because if the original address is occupied, we search for another open address.

Example:

```
Try index 5
If full, try index 6
If full, try index 7
...
```

This idea will be studied more in later phases.

20. Code Example: Ideal Hashing in C++

```
#include <iostream>
using namespace std;

int main() {
    int hashTable[20];

    for (int i = 0; i < 20; i++) {
        hashTable[i] = -1;
    }

    int keys[] = {8, 3, 6, 10, 15, 18, 4};
    int numKeys = 7;

    for (int i = 0; i < numKeys; i++) {
        int key = keys[i];
        int index = key; // H(x) = x
        hashTable[index] = key;
    }
}
```



```
int searchKey = 10;
int searchIndex = searchKey; // H(x) = x

if (hashTable[searchIndex] == searchKey) {
    cout << "Key found at index " << searchIndex << endl;
} else {
    cout << "Key not found" << endl;
}

return 0;
}
```

21. Explanation of the Ideal Hashing Code

Important parts

```
int hashTable[20];
```

This creates a hash table of size 20.

```
hashTable[i] = -1;
```

This sets all positions to `-1`.

Here, `-1` means empty.

```
int index = key;
```

This is the hash function:

$$H(x) = x$$

So the key itself becomes the index.

```
hashTable[index] = key;
```

This stores the key at the calculated index.

```
int searchIndex = searchKey;
```

To search, we again use:

$$H(x) = x$$

So if the search key is 10, we check index 10 directly.

22. Dry Run of Ideal Hashing Code

Keys:

8, 3, 6, 10, 15, 18, 4

Using:

$$H(x) = x$$

Dry run:

```
Insert 8  at index 8
Insert 3  at index 3
Insert 6  at index 6
Insert 10 at index 10
Insert 15 at index 15
Insert 18 at index 18
Insert 4  at index 4
```

To search for 10:

```
Check index 10
Index 10 contains 10
So 10 is found immediately
```

23. Code Example: Modulus Hashing in C++

```
#include <iostream>
using namespace std;

int hashFunction(int key) {
    return key % 10;
}

int main() {
    int hashTable[10];

    for (int i = 0; i < 10; i++) {
        hashTable[i] = -1;
    }

    int keys[] = {8, 3, 6, 10, 15};
    int numKeys = 5;

    for (int i = 0; i < numKeys; i++) {
        int key = keys[i];
        int index = hashFunction(key);
        hashTable[index] = key;
    }

    int searchKey = 15;
    int searchIndex = hashFunction(searchKey);

    if (hashTable[searchIndex] == searchKey) {
        cout << "Key found at index " << searchIndex << endl;
    }

    return 0;
}
```

24. Explanation of the Modulus Hashing Code

Hash function

```
int hashFunction(int key) {
    return key % 10;
}
```

This function takes a key and returns an index from 0 to 9.

Example:

```
hashFunction(15) = 15 % 10 = 5
```

Hash table

```
int hashTable[10];
```

This creates a table of size 10.

Valid indices are:

```
0 to 9
```

Empty marker

```
hashTable[i] = -1;
```

The value -1 means the position is empty.

Insert process

```
int index = hashFunction(key);  
hashTable[index] = key;
```

For each key:

1. Calculate the index.
 2. Store the key at that index.
-

25. Dry Run of Modulus Hashing Code

Keys:

8, 3, 6, 10, 15

Hash function:

$H(x) = x \bmod 10$

Dry run:

```
8 mod 10 = 8 → store 8 at index 8
3 mod 10 = 3 → store 3 at index 3
6 mod 10 = 6 → store 6 at index 6
10 mod 10 = 0 → store 10 at index 0
15 mod 10 = 5 → store 15 at index 5
```

Final table:

```
Index 0: 10
Index 1: empty
Index 2: empty
Index 3: 3
Index 4: empty
Index 5: 15
Index 6: 6
Index 7: empty
Index 8: 8
Index 9: empty
```

Search for 15:

```
15 mod 10 = 5
Check index 5
Index 5 contains 15
So 15 is found
```

26. Important Limitation of This Code

The modulus hashing code ignores collisions.

That means if we insert 25:

$$25 \bmod 10 = 5$$

Index 5 already has 15.

The simple code does not properly solve this.

This is why we need later phases.

27. Complete Phase 1 Example Program

```
#include <iostream>
using namespace std;

const int IDEAL_SIZE = 20;
const int MOD_SIZE = 10;

int modHash(int key) {
    return key % MOD_SIZE;
}

void printTable(const int table[], int size) {
    for (int i = 0; i < size; i++) {
        cout << i << ": ";
        if (table[i] == -1) {
            cout << "empty";
        } else {
            cout << table[i];
        }
        cout << endl;
    }
}

int main() {
    int idealTable[IDEAL_SIZE];
    int modTable[MOD_SIZE];

    for (int i = 0; i < IDEAL_SIZE; i++) {
```

```

        idealTable[i] = -1;
    }

    for (int i = 0; i < MOD_SIZE; i++) {
        modTable[i] = -1;
    }

    int keys[] = {8, 3, 6, 10, 15, 18, 4};
    int n = 7;

    cout << "Ideal hashing / direct addressing" << endl;

    for (int i = 0; i < n; i++) {
        int key = keys[i];
        idealTable[key] = key;
    }

    printTable(idealTable, IDEAL_SIZE);

    cout << "\nModulus hashing, ignoring collisions for now" << endl;

    for (int i = 0; i < 5; i++) {
        int key = keys[i];
        int index = modHash(key);
        modTable[index] = key;
    }

    printTable(modTable, MOD_SIZE);

    int collisionKey = 25;

    cout << "\nCollision example: " << collisionKey
        << " maps to index " << modHash(collisionKey)
        << ", where 15 already maps." << endl;

    return 0;
}

```

28. Explanation of the Complete Program

This program shows two ideas:

1. Ideal hashing / direct addressing
2. Modulus hashing

First part

```
idealTable[key] = key;
```

This stores the key directly at the same index.

Example:

Key 10 goes to index 10

Second part

```
int index = modHash(key);  
modTable[index] = key;
```

This uses modulus hashing.

Example:

Key 15 goes to index 5

Collision part

```
int collisionKey = 25;
```

The program shows:

$25 \bmod 10 = 5$

But 15 already maps to index 5.

So this shows the collision problem.

29. Time Complexity in Phase 1

29.1 Direct addressing

Insertion:

$O(1)$

Search:

$O(1)$

Reason:

The key directly gives the index.

29.2 Modulus hashing

Calculating the index is also:

$O(1)$

Example:

$25 \bmod 10 = 5$

This calculation is fast.

But collisions can make the full hash table operation more complicated.

So modulus hashing needs a collision-resolution method.

30. Common Beginner Mistakes

30.1 Mistake: Confusing key and index

A key is the value we store or search.

An index is the position in the array.

In ideal hashing:

Key 15 → Index 15

But in modulus hashing:

Key 15 → Index 5

So key and index are not always the same.

30.2 Mistake: Thinking modulus removes collisions

Wrong idea:

Modulus hashing means no collisions.

Correct idea:

Modulus hashing keeps indices inside the array, but collisions can still happen.

Example:

$15 \bmod 10 = 5$
 $25 \bmod 10 = 5$

Both map to index 5.

30.3 Mistake: Forgetting array bounds

If the table size is 10, valid indices are:

0 to 9

Index 10 is invalid.

Using modulus helps prevent invalid indices.

Example:

```
key % 10
```

will always produce a value from 0 to 9.

30.4 Mistake: Using 0 or -1 carelessly as empty markers

In examples, -1 is often used to mean empty.

Example:

```
Index 2: -1 means empty
```

But this only works if -1 is not a valid key.

If -1 can be a real key, then using it as an empty marker causes confusion.

Similarly, some programs use 0 to mean empty.

But if 0 is a real key, this becomes a problem.

A safer solution is to use a separate status array or choose an empty marker that cannot be a real key.

31. Very Important Formulas

Ideal hashing

```
H(x) = x
```

Meaning:

```
The key itself is the index.
```

Example:

$$H(8) = 8$$

Modulus hashing

$$H(x) = x \bmod \text{size}$$

Meaning:

Divide x by the table size and use the remainder as the index.

Example with size 10:

$$H(25) = 25 \bmod 10 = 5$$

32. Phase 1 Full Summary

Phase 1 teaches the foundation of hashing.

The main problem is searching.

Linear search is simple but can be slow because it checks values one by one.

Binary search is faster, but it needs sorted data.

Hashing tries to be faster by calculating where a key should be stored.

A hash function converts a key into an index.

A hash table is the array where keys are stored.

Ideal hashing uses:

$$H(x) = x$$

This is fast because the key directly gives the index.

But ideal hashing can waste memory because the array size depends on the largest possible key.

To reduce memory waste, we use modulus hashing:

$$H(x) = x \bmod \text{size}$$

This keeps indices inside a fixed-size table.

But modulus hashing can create collisions.

A collision happens when two keys map to the same index.

Example:

$$\begin{aligned} 15 \bmod 10 &= 5 \\ 25 \bmod 10 &= 5 \end{aligned}$$

Both keys want index **5**.

To handle collisions, we need collision-resolution methods.

The two main families are:

1. Open hashing / chaining
2. Closed hashing / open addressing

Chaining stores colliding keys in a linked list.

Open addressing searches for another empty place inside the same table.

These methods are studied in later phases.

33. Phase 1 Quick Revision Sheet

Key idea

Hashing = calculate the position instead of searching one by one.

Main terms

Term	Meaning
Key	Value to store/search
Index	Array position
Hash table	Array used for hashing
Hash function	Formula that converts key to index
Domain	Possible input keys
Range	Possible output indices
Collision	Two keys map to the same index

Main formulas

Ideal hashing: $H(x) = x$
Modulus hashing: $H(x) = x \bmod \text{size}$

Example

$H(15) = 15 \bmod 10 = 5$
 $H(25) = 25 \bmod 10 = 5$

So 15 and 25 collide at index 5.

Why Phase 1 matters

Phase 1 prepares you for all later hashing topics.

Before solving collisions, you must understand how collisions happen.

That is why Phase 1 ends with collision handling families.

34. What You Should Be Able to Explain After Phase 1

After studying this phase, you should be able to explain:

1. Why linear search can be slow
2. Why binary search needs sorted data
3. Why hashing is introduced
4. What a key is

5. What an index is
 6. What a hash table is
 7. What a hash function is
 8. How ideal hashing works
 9. Why ideal hashing wastes memory
 10. How modulus hashing works
 11. Why modulus keeps indices inside the table
 12. What a collision is
 13. Why collisions happen
 14. What open hashing means
 15. What closed hashing means
 16. Why the next phases are needed
-

35. Final Simple Explanation

Hashing is like using a formula to decide where data should go.

Instead of searching the whole array, we calculate the position.

Ideal hashing is the easiest form:

$$\text{Key} = \text{Index}$$

But it wastes memory.

Modulus hashing fixes the memory problem by using a smaller table:

$$\text{Index} = \text{Key} \bmod \text{Table Size}$$

But this creates a new problem: collisions.

A collision happens when two keys want the same index.

So the next step is learning how to solve collisions.

That is exactly what Phase 2 begins with: chaining.